JML Operations Log

Project 3: PowerShell Automation for Active Directory

Author: Mohd. Ismayeel Khan | **Date:** 12 and 14 October 2026 | **Environment:** VMware Workstation 17 Player | Windows Server 2022 | Windows PowerShell 5.1 | Domain: Project.local

Scripts covered: New-BulkUsers.ps1 (Joiner) | Invoke-Offboarding.ps1 (Leaver) | Invoke-Mover.ps1 (Mover)

About this log

This is the record of every account operation I ran in the lab: one batch of new hires, two leavers and two movers. I wrote it the way I would log tickets in a helpdesk queue, so every entry follows the same five steps:

1. **Request:** what was asked for.
2. **Plan:** a dry run to see what would happen.
3. **Execute:** the real run.
4. **Verify:** check the result in PowerShell and in Active Directory Users and Computers.
5. **Result and lessons:** what happened and what I took from it.

The requests are simulated. I wrote them the way an HR or line-manager request would arrive. The ticket numbers (TKT-1001 and so on) are recorded in the CSV files and, for leavers, in the Description field of each disabled account. The proof that these scripts are safe to re-run, and how they handle broken input, is in Document 3.

Entry	Operation	Date	Script	Result
Batch	Joiner: 6 new hires	12 Oct 2026	New-BulkUsers.ps1	6 accounts created
TKT-1001	Leaver: Neha Joshi, resignation	12 Oct 2026	Invoke-Offboarding.ps1	Offboarded

Entry	Operation	Date	Script	Result
TKT-1002	Leaver: Karan Malhotra, end of contract	12 Oct 2026	Invoke-Offboarding.ps1	Offboarded
TKT-2001	Mover: Aarav Mehta, IT to Management	14 Oct 2026	Invoke-Mover.ps1	Moved
TKT-2002	Mover: Priya Nair, HR to Finance	14 Oct 2026	Invoke-Mover.ps1	Moved

Part A: Joiner, onboarding six new hires

The request

HR (simulated) sent six new starters across four departments. No ticket number was recorded for this batch. I saved them in NewHires.csv:

First name	Last name	Department	Job title
Aarav	Mehta	IT	Support Technician
Karan	Malhotra	IT	Junior System Administrator
Priya	Nair	HR	HR Executive
Rohan	Verma	Finance	Accounts Analyst
Neha	Joshi	Finance	Payroll Specialist
Sneha	Kapoor	Management	Operations Manager

What the script does

For every row in the CSV, New-BulkUsers.ps1:

1. Builds a username from the first initial and the last name, in lowercase (Aarav Mehta becomes amehta).
2. Looks up the department in Config.ps1 to find the right OU and security group.
3. Skips the person if that username already exists, so running the script twice is safe.
4. Generates a random temporary password that meets the domain's complexity rules.
5. Creates the account in the right OU and forces a password change at first sign-in.

6. Adds the account to the department's security group.
7. Writes a log, a report and a separate hand-off file for the temporary passwords.

On passwords: the temporary passwords never appear in the log or in the report. They go into a separate hand-off file, a bit like a sealed envelope for the new hire's manager. That file is never shown in a screenshot and never uploaded to GitHub.

Step 1: Plan (dry run)

I always dry-run first. The -WhatIf switch means "pretend": the script shows exactly what it would do and changes nothing.

```
cd C:\PowerShell-AD-Automation\Scripts
.\New-BulkUsers.ps1 -WhatIf
```

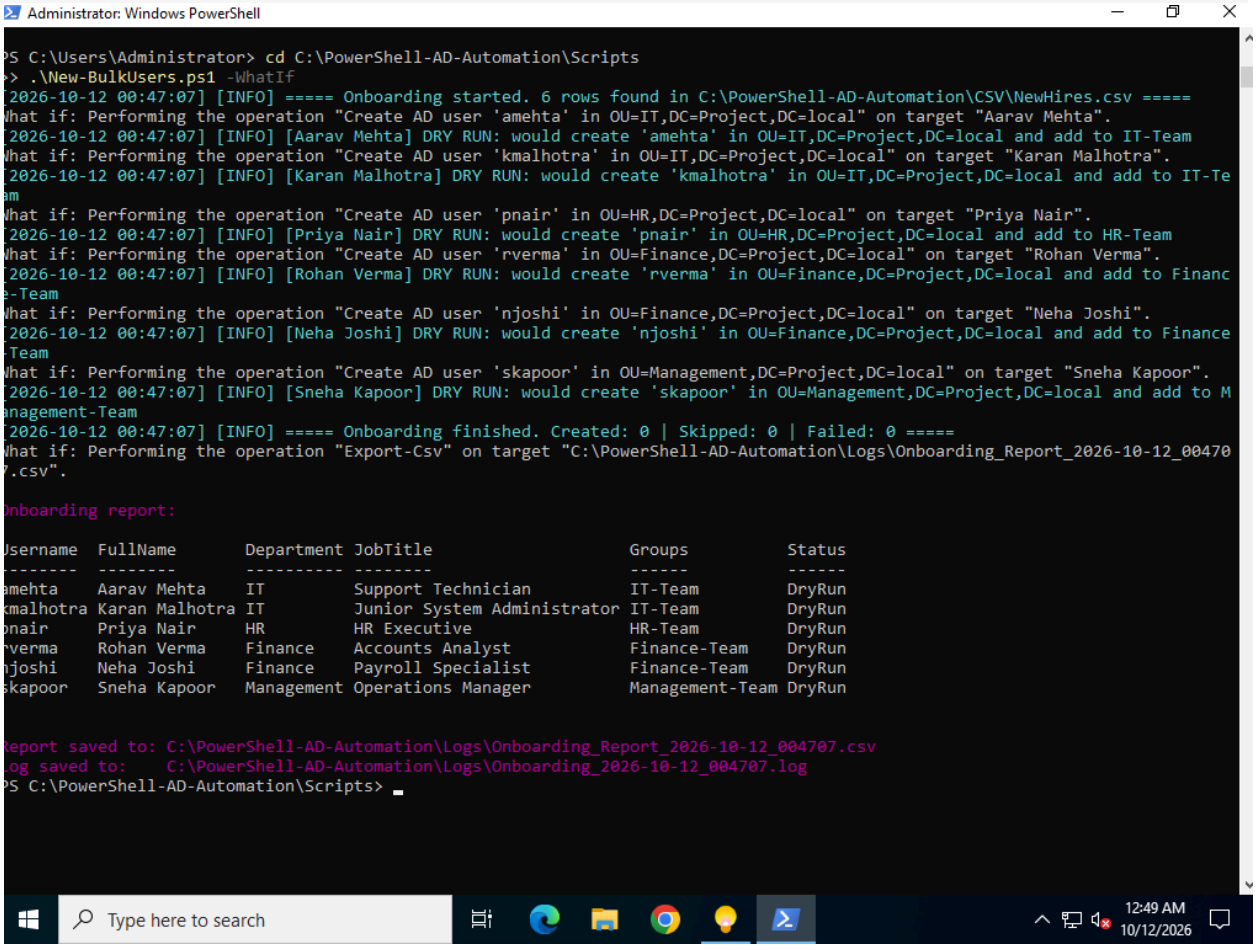


Figure 1: Dry run of the onboarding script. Six accounts are planned and nothing is created.

Each DRY RUN line names the username, the OU the account would go into and the group it would join. The table at the bottom lists all six people with the status DryRun, and the summary line reads Created: 0, so nothing was created.

Step 2: Execute

With the plan checked, I ran the script for real:

```
.\New-BulkUsers.ps1
```

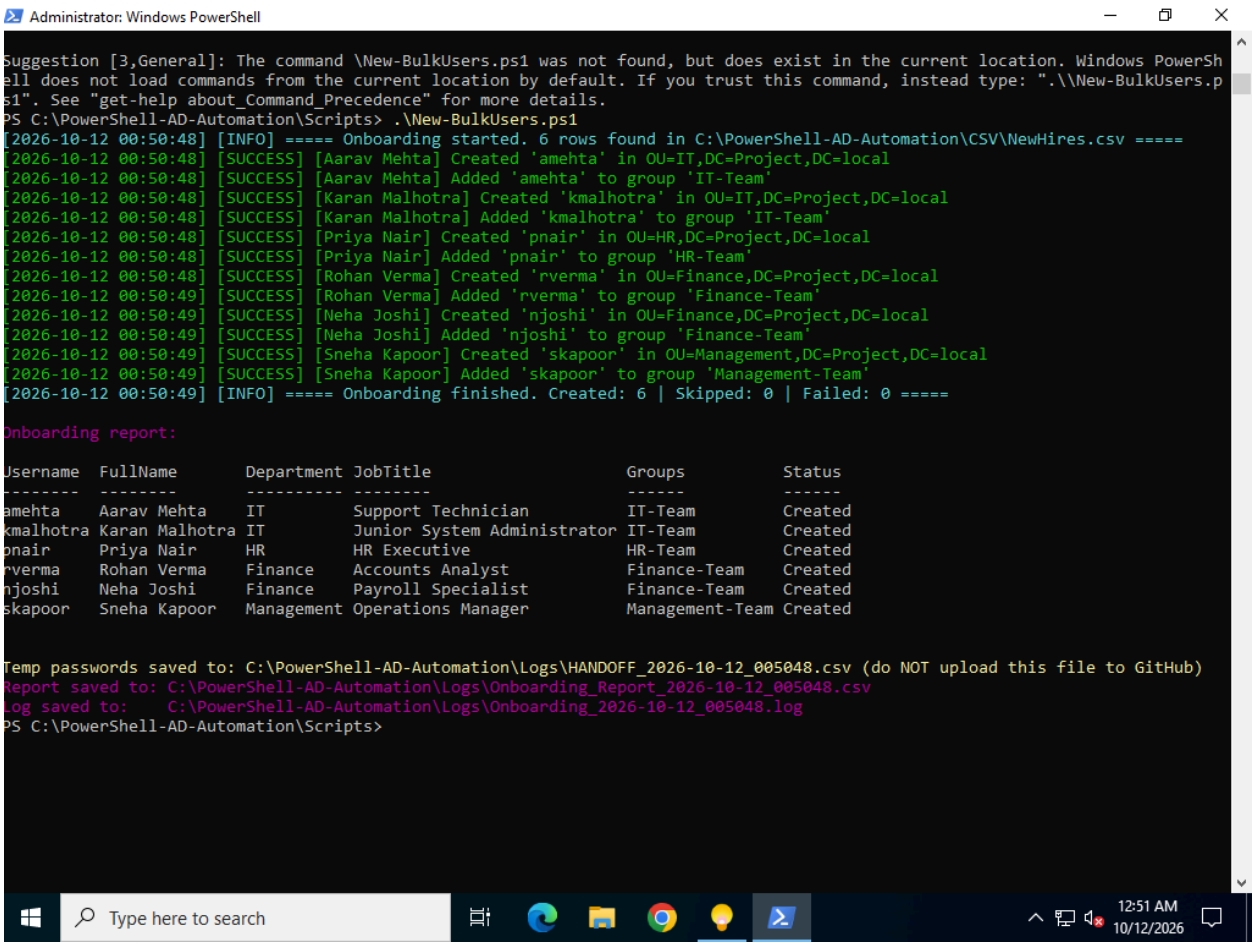


Figure 2: Real run. Six accounts created in the right OUs and groups.

Every user gets two green SUCCESS lines: one for creating the account in its OU and one for adding it to its group. The report shows all six with the status Created, and the summary line reads Created: 6 | Skipped: 0 | Failed: 0. The script also saved the hand-off file, the report and the log (only the file names are visible on screen, no passwords).

The grey message at the very top is from my first attempt. I typed the command without .\ in front of it, and PowerShell told me the script exists in the current folder and suggested the fix. I left the screenshot as it was.

Step 3: Verify

I don't trust a script's own report on its own, so I checked the result in two ways.

Check 1, in Active Directory Users and Computers. I opened the IT OU. Aarav Mehta and Karan Malhotra are there next to the four IT users that were already in the lab (Alena Costa, Collette Torres, John Smith and Sarah Conner), along with the IT-Team group itself.

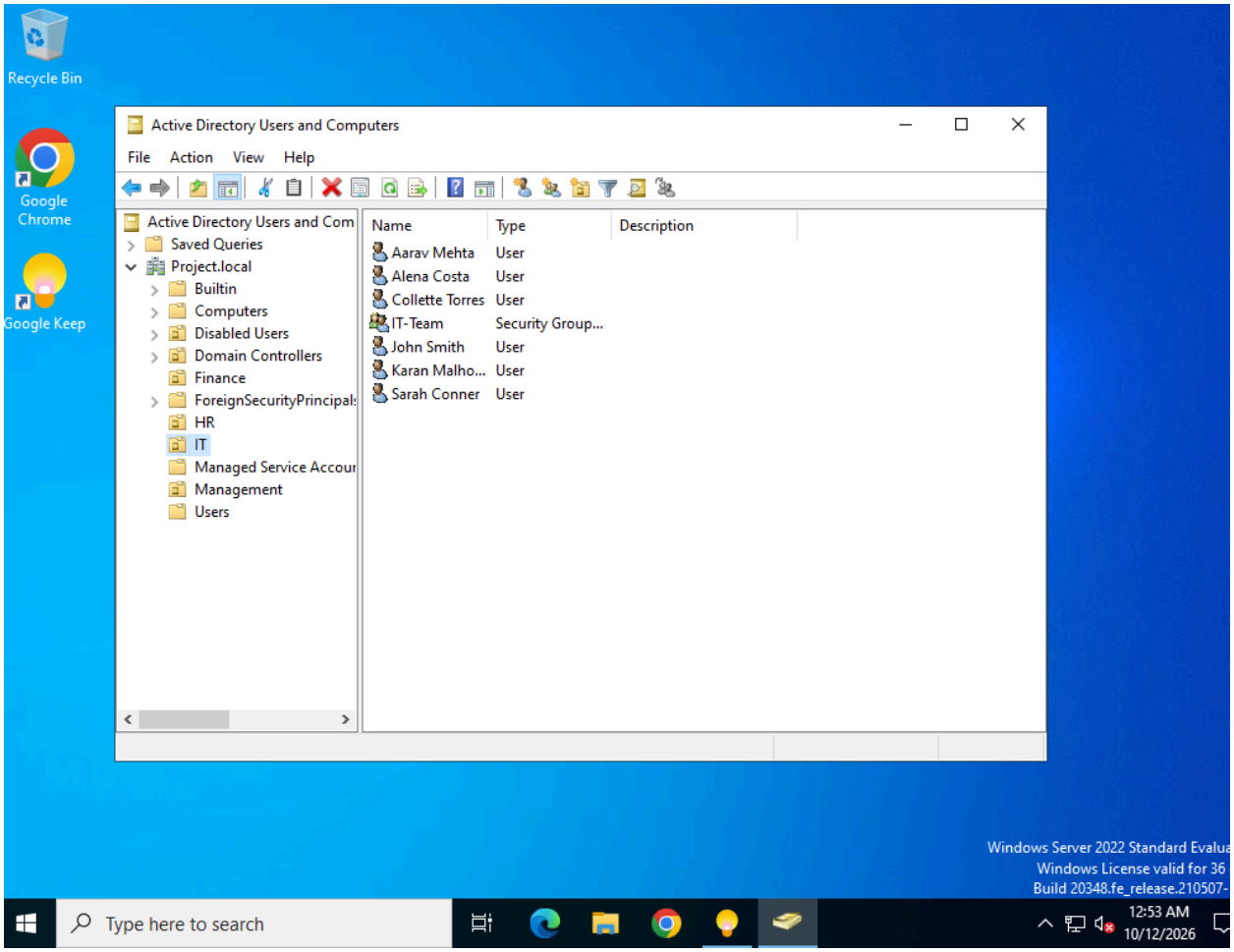


Figure 3: Active Directory Users and Computers showing the new IT users inside the IT OU.

Check 2, in PowerShell. I listed the members of the IT-Team group:

```
Get-ADGroupMember -Identity "IT-Team" | Select-Object Name,
SamAccountName
```

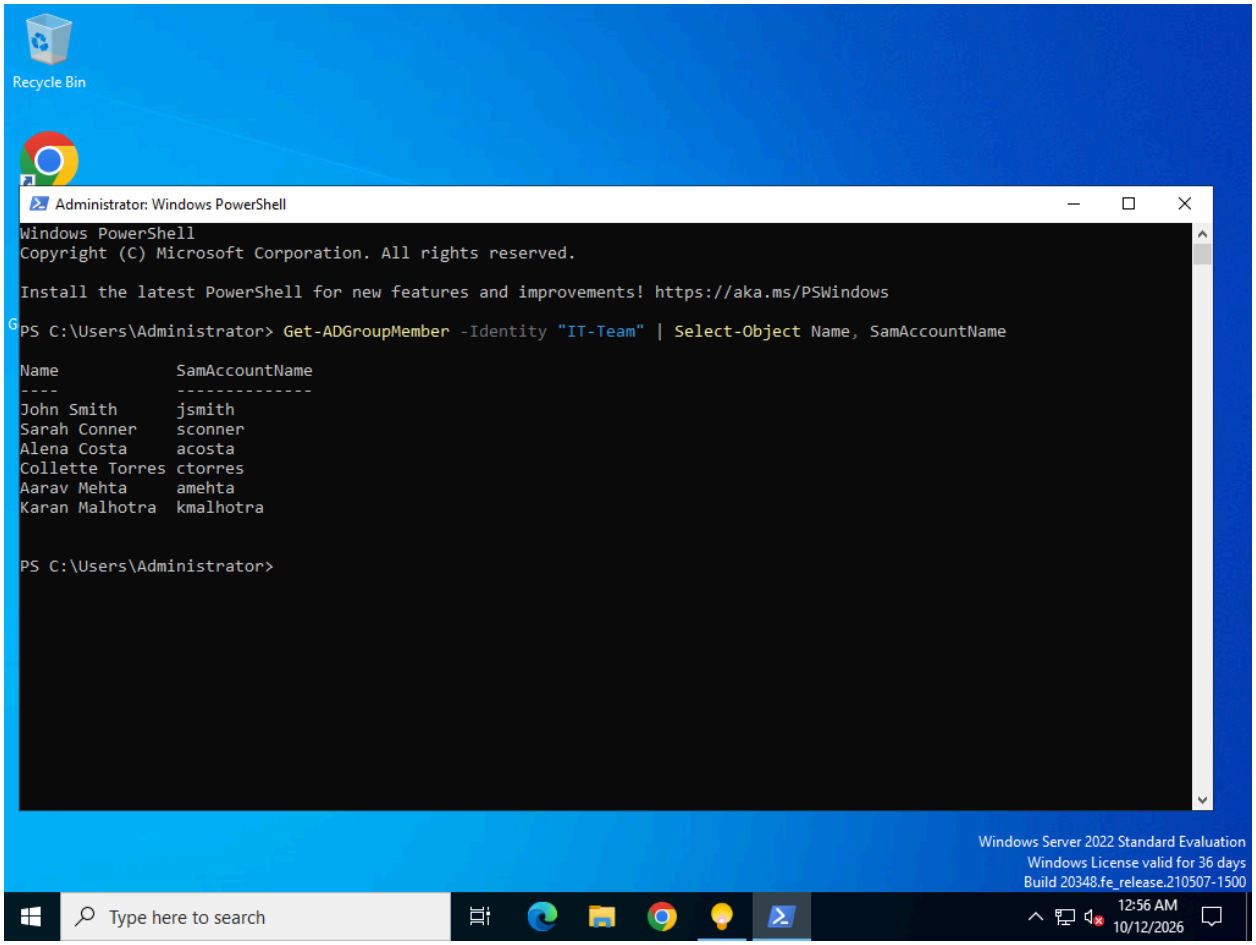


Figure 4: IT-Team now has six members: the four original users plus amehta and kmalhotra.

Result and lessons

- Six accounts were created, each in the correct OU and security group, with a temporary password and a forced change at first sign-in.
- The dry run showed me the whole plan before anything changed, which is the habit I want to keep.
- The temporary passwords stay in a separate hand-off file, away from reports and screenshots.
- Checking in both PowerShell and Active Directory Users and Computers caught nothing wrong this time, but it is a cheap way to make sure the script and the directory agree.

Part B: Leavers, offboarding two users

The requests

Ticket	Employee	Department	Reason
TKT-1001	Neha Joshi (njoshi)	Finance	Resignation
TKT-1002	Karan Malhotra (kmalhotra)	IT	End of contract

The input file, Leavers.csv, looks like this:

```
Username,Reason,TicketRef
njoshi,Resignation,TKT-1001
kmalhotra,End of contract,TKT-1002
```

I used two accounts from the batch above on purpose, so my original lab accounts were never at risk.

What the script does (and why the order matters)

Before it changes anything, Invoke-Offboarding.ps1 refuses protected accounts (Administrator, Guest and krbtgt) and members of Domain Admins, and it skips anyone who does not exist or who is already in the Disabled Users OU. For everyone else it works in this order:

1. Records the before state (the current OU and groups) for the report.
2. **Disables the account first**, so access stops immediately even if a later step fails.
3. Resets the password to a random value that nobody knows and that is never stored.
4. Removes every group membership.
5. Writes the date, reason and ticket number into the account's Description.
6. **Moves the account to the Disabled Users OU last**, because moving changes the account's path.
7. Writes a log and a deprovisioning report.

The Description it writes looks like this:

```
Disabled 2026-10-12 | Reason: Resignation | Ticket: TKT-1001
```

The account is disabled and kept, never deleted. Companies keep disabled accounts for a retention period so they are available for audits, recovery and mistakes.

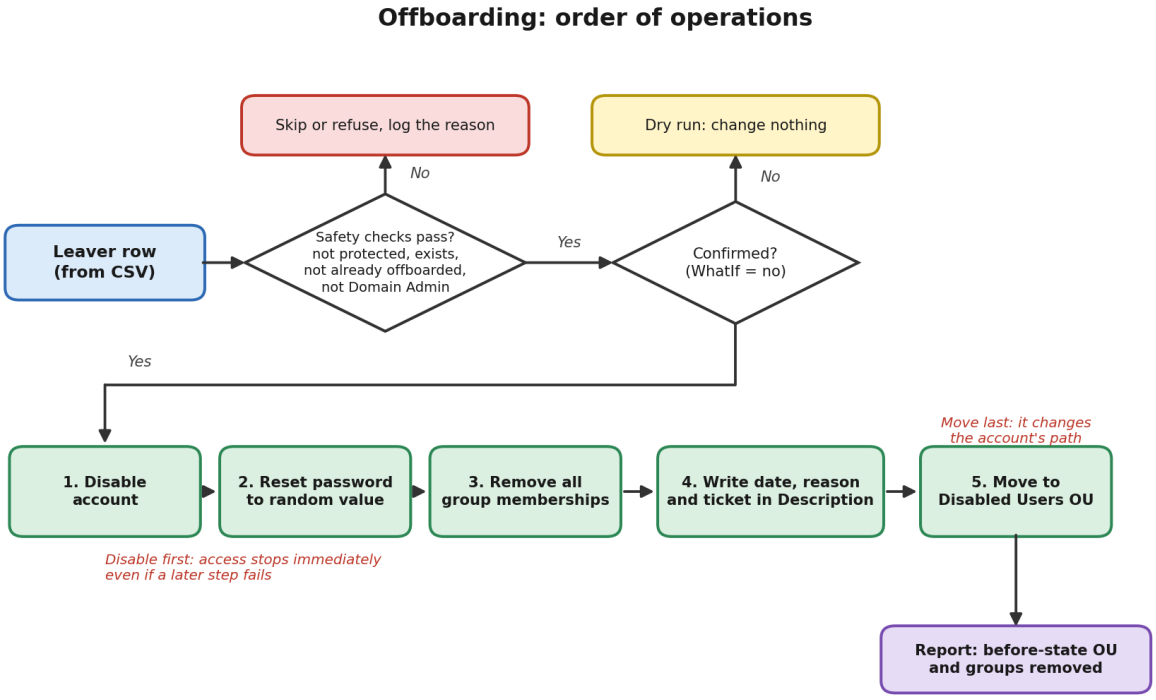


Figure 5: The order of operations during offboarding. Disable first, move last.

Step 1: Capture the before state

A before screenshot is what makes the after screenshot meaningful, so I captured the account's state and the IT-Team members first:

```
Get-ADUser njoshi -Properties MemberOf, Enabled | Select-Object Name,
Enabled, DistinguishedName, MemberOf
Get-ADGroupMember -Identity "IT-Team" | Select-Object Name,
SamAccountName
```

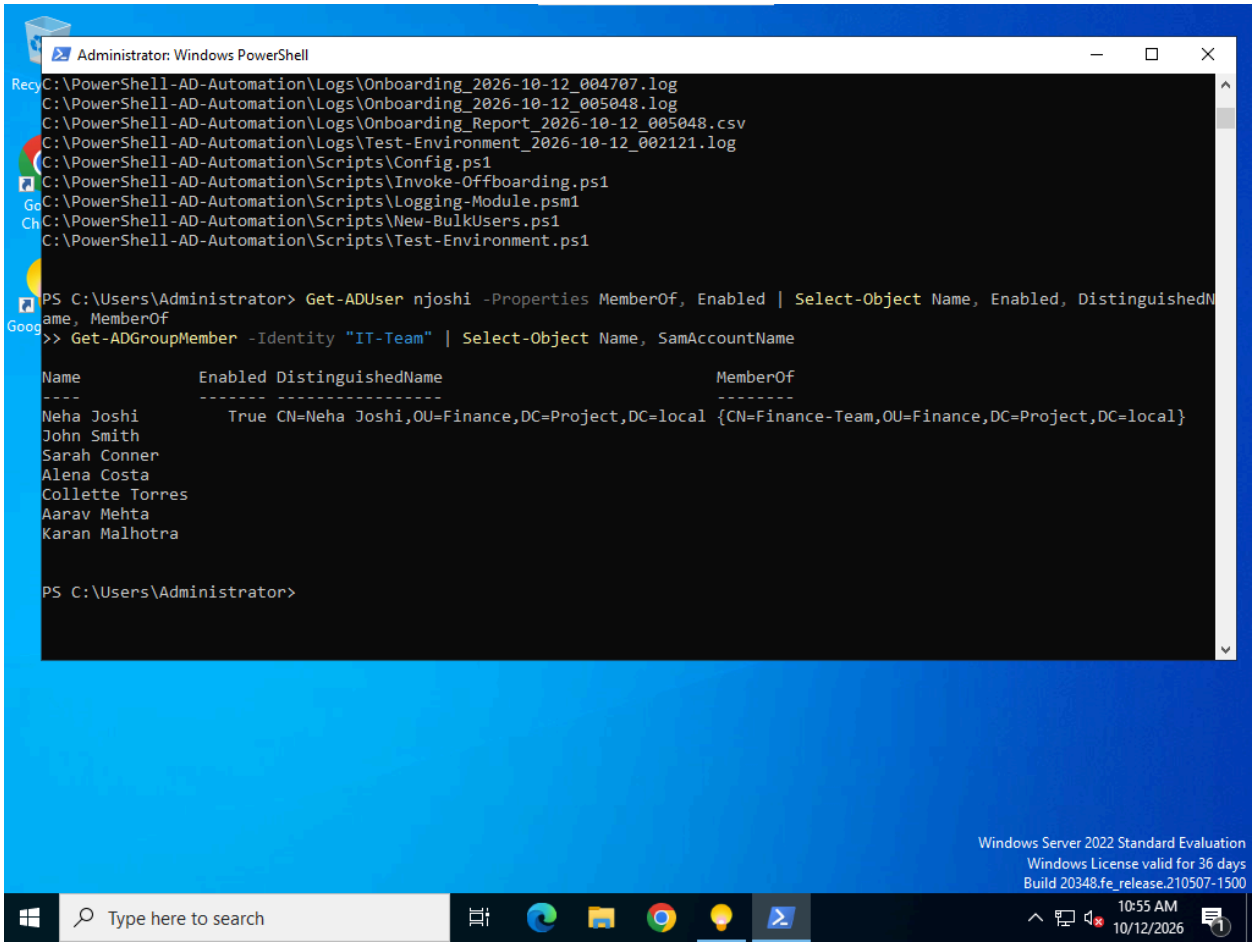


Figure 6: Before offboarding. Neha Joshi is enabled, sits in the Finance OU and belongs to Finance-Team. Karan Malhotra is still in IT-Team.

Step 2: Plan (dry run)

```
.\Invoke-Offboarding.ps1 -WhatIf
```

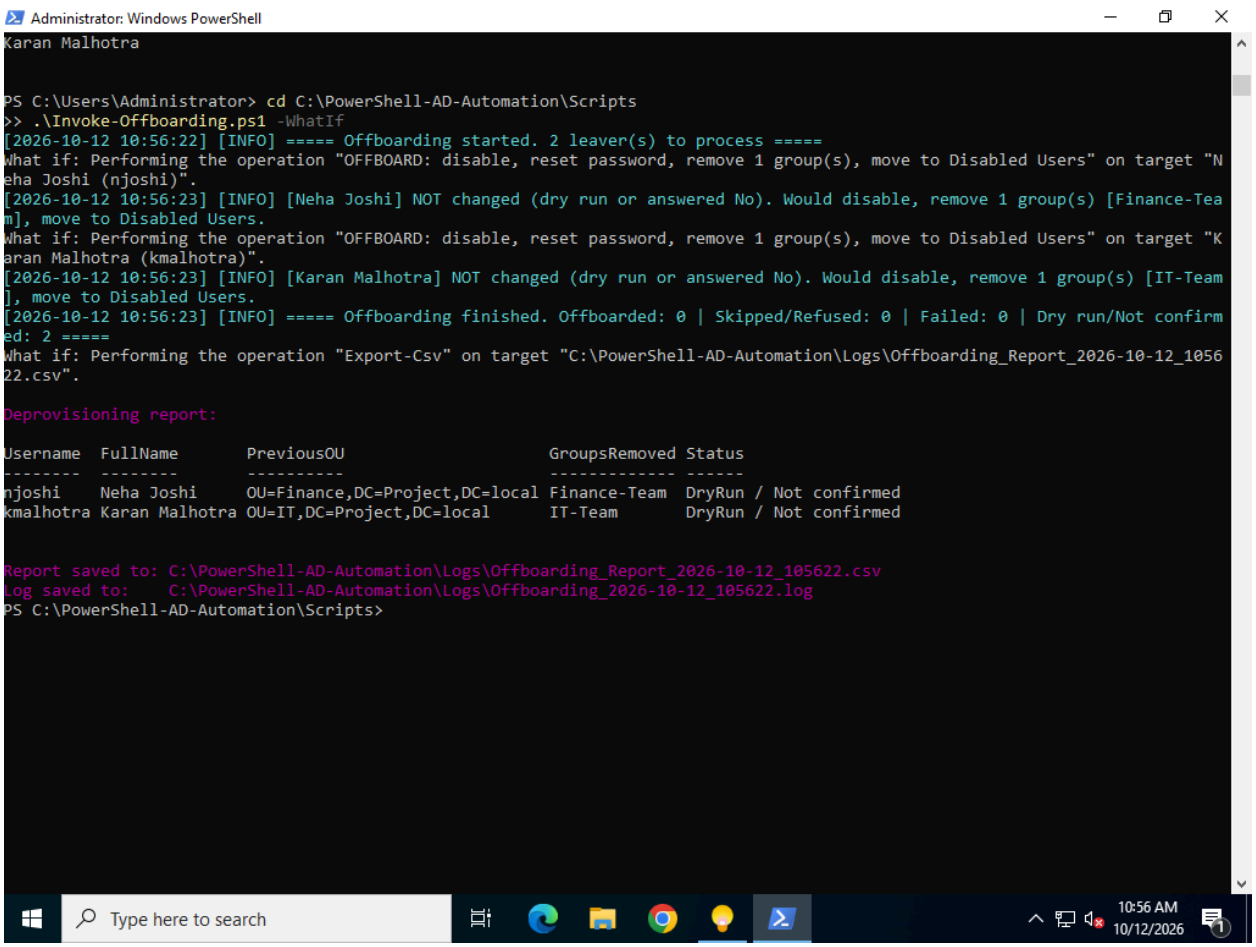


Figure 7: Offboarding dry run. The script says what it would do and changes nothing.

For each user the log says the account was NOT changed and what it would have done: disable the account, remove one group (Finance-Team for Neha, IT-Team for Karan) and move it to Disabled Users. The report shows each person's previous OU and the group that would be removed, with the status DryRun / Not confirmed.

Step 3: Execute, with confirmation

```
.\Invoke-Offboarding.ps1
```

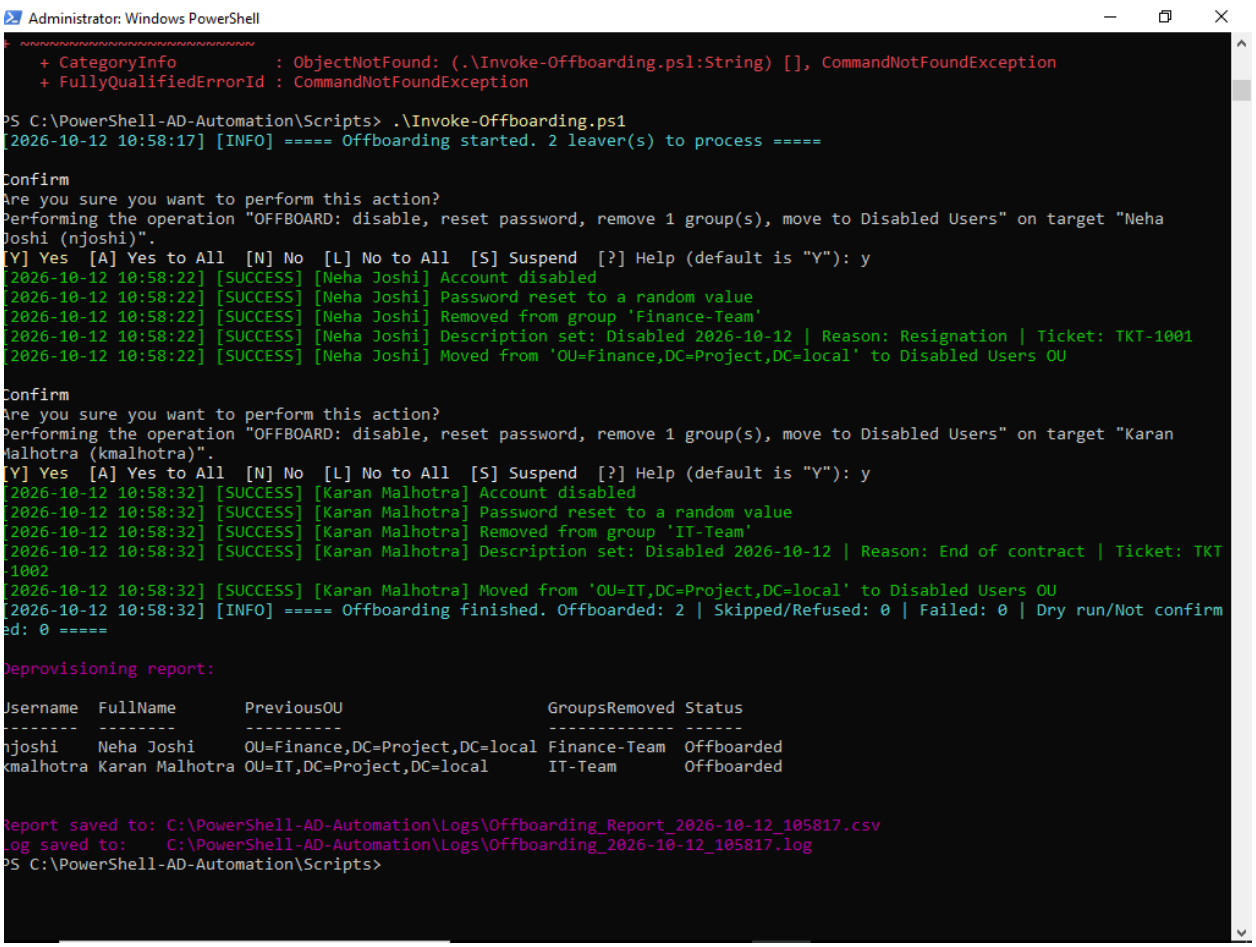


Figure 8: The Yes/No confirmation prompt for each user, followed by every offboarding step and the deprovisioning report.

Offboarding is destructive, so the script stops and asks "Are you sure you want to perform this action?" for every user. I answered Y each time. After each confirmation the log shows the steps in order: account disabled, password reset to a random value, removed from its group, Description set with the reason and ticket, and moved to the Disabled Users OU. The summary line reads Offboarded: 2 | Skipped/Refused: 0 | Failed: 0.

The red text at the top of the screenshot is a typo from an earlier attempt (I typed the script name with the wrong extension). I kept it as it was.

Step 4: Verify the after state

```
Get-ADUser njoshi -Properties Enabled, Description, MemberOf |
Select-Object Name, Enabled, Description, DistinguishedName, MemberOf
Get-ADGroupMember -Identity "IT-Team" | Select-Object Name,
SamAccountName
```

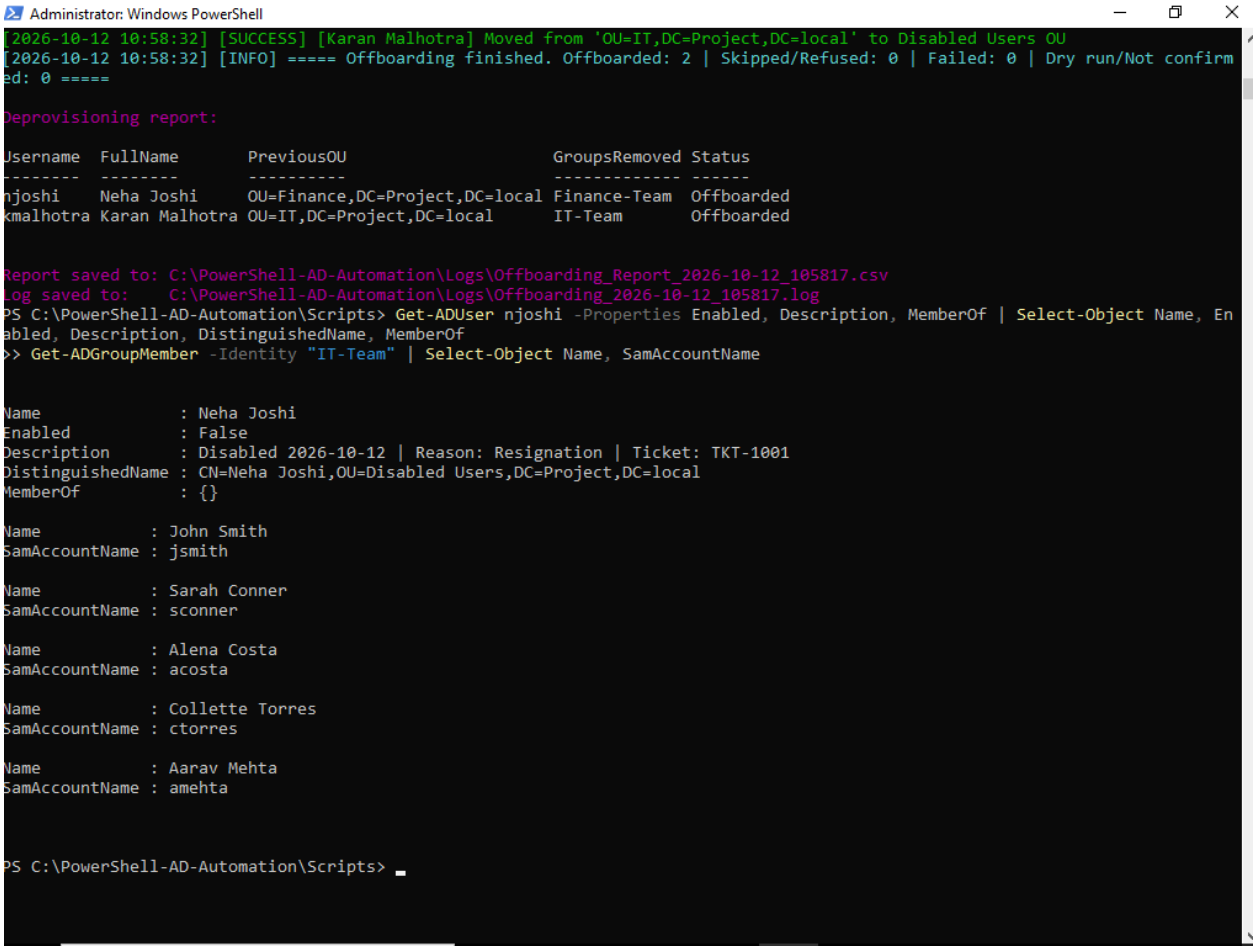


Figure 9: After offboarding. The account is disabled, its Description records the reason and ticket, it has no group memberships, and Karan Malhotra is gone from IT-Team.

The account now shows Enabled as False, the Description reads "Disabled 2026-10-12 | Reason: Resignation | Ticket: TKT-1001", the distinguished name sits in the Disabled Users OU and MemberOf is empty. IT-Team now lists five people instead of six.

I also checked the Disabled Users OU in Active Directory Users and Computers:

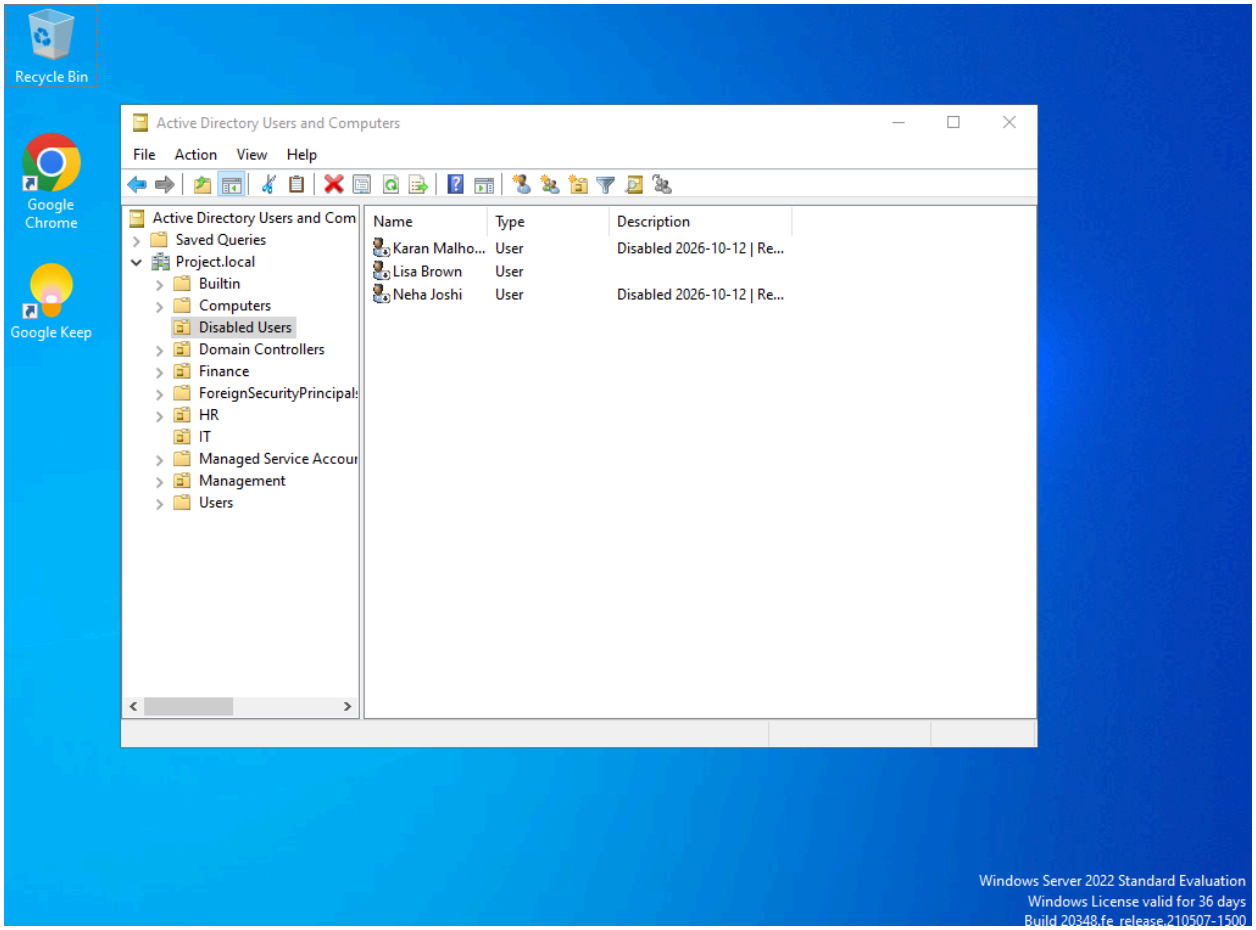


Figure 10: The offboarded accounts inside the Disabled Users OU, with the reason and date visible in the Description column.

Karan Malhotra and Neha Joshi are both there with a Description that starts with "Disabled 2026-10-12". The OU also contains Lisa Brown, an account that is not part of this run.

Step 5: Re-run

I ran the script a second time to prove it will not process the same people twice:

```
.\Invoke-Offboarding.ps1
```

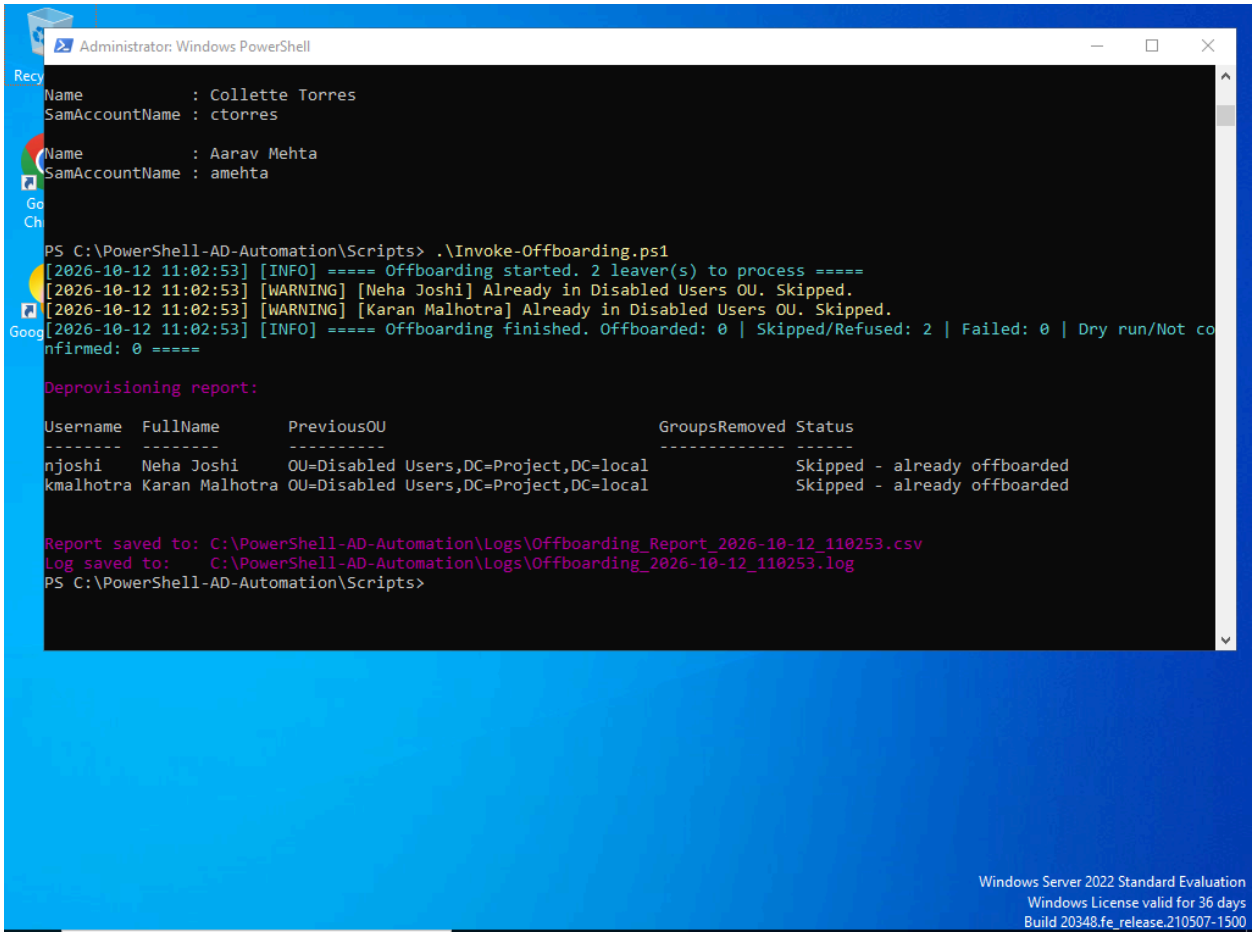


Figure 11: Re-running offboarding. Both users are skipped because they are already in the Disabled Users OU.

Both users show the warning "Already in Disabled Users OU. Skipped." and the status Skipped - already offboarded. The summary reads Offboarded: 0 | Skipped/Refused: 2.

Result and lessons

- Both accounts were disabled, had their passwords reset, lost all group memberships, were annotated with the reason and ticket, and were moved to Disabled Users.
- Disabling first and moving last means access stops immediately and a failure part-way cannot leave an account half-open.
- Disabling instead of deleting keeps the account available for audits and recovery.
- The confirmation prompt is the safety net for a destructive action, and the report records which groups each person had before removal, which is the evidence an auditor wants.

Part C: Movers, changing two users' departments

The requests

Ticket	Employee	Move	New job title
TKT-2001	Aarav Mehta (amehta)	IT to Management	IT Operations Lead
TKT-2002	Priya Nair (pnair)	HR to Finance	Payroll Coordinator

The input file, Movers.csv:

```
Username,NewDepartment,NewJobTitle,TicketRef
amehta,Management,IT Operations Lead,TKT-2001
pnair,Finance,Payroll Coordinator,TKT-2002
```

Why movers matter

The Mover stage is the one people forget. When someone changes department it is easy to give them the new access and never take the old access away. They end up holding both, and over the years it piles up. This is called privilege creep, and it is one of the most common findings in access audits. The Mover script exists to prevent it.

What the script does

Before it changes anything, Invoke-Mover.ps1 refuses protected accounts, disabled accounts and unknown departments, and it skips anyone who is already in the new department's OU. For everyone else it works in this order:

1. **Adds the new department's group first**, so there is never a gap in the person's access.
2. **Removes the old department's group**, so they only keep what their new role needs (least privilege).
3. Leaves any groups that are not department groups alone, but logs them for manager review. For example, a project group would be kept and flagged.
4. Updates the Department and Job Title on the account.
5. **Moves the account to the new department's OU last**, because moving changes the account's path.
6. Writes a log and a mover report.

Step 1: Plan (dry run)

```
.\Invoke-Mover.ps1 -WhatIf
```

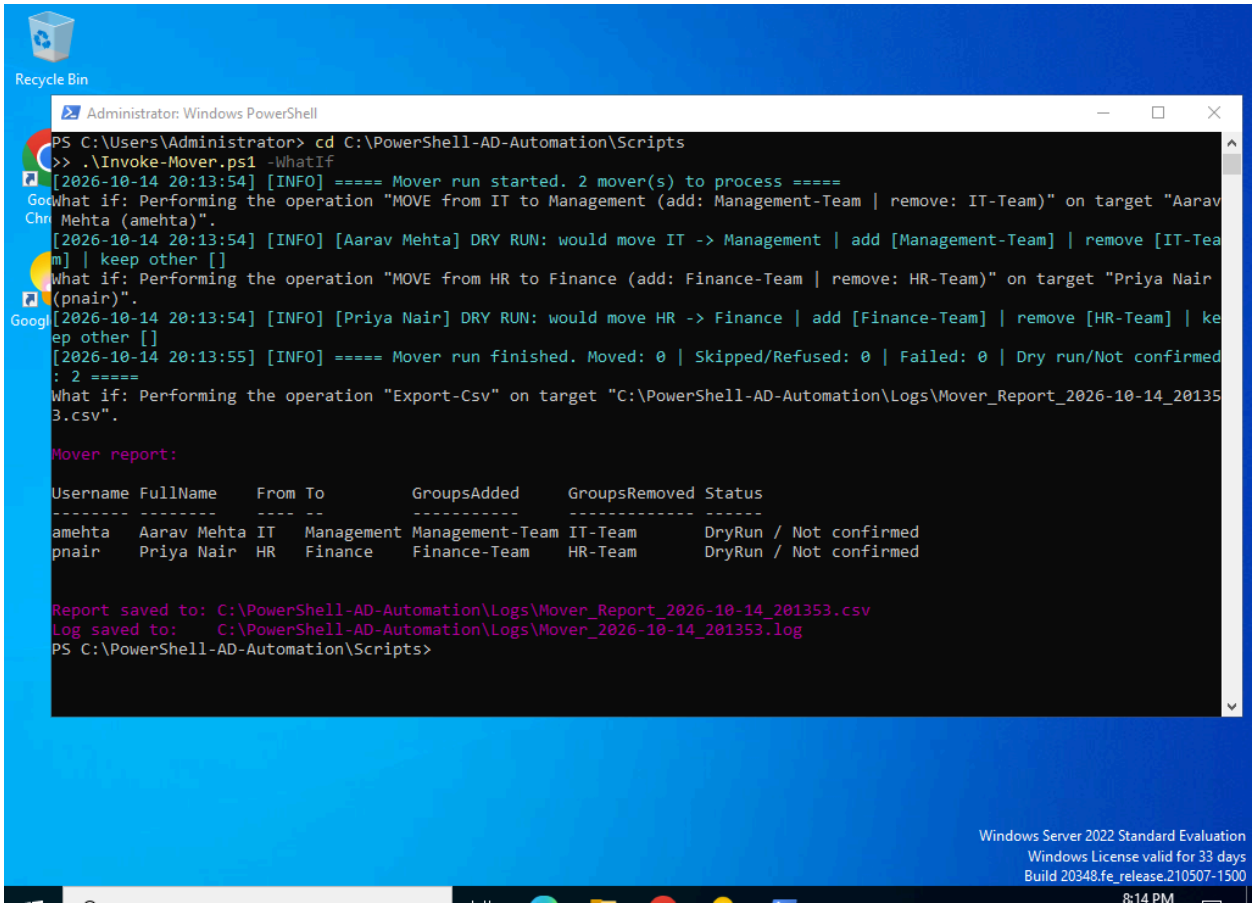


Figure 12: Mover dry run. The script reports the planned moves and group changes without making any.

The log lines spell out the plan for each person: Aarav would move from IT to Management, gaining Management-Team and losing IT-Team, and Priya would move from HR to Finance, gaining Finance-Team and losing HR-Team. The empty brackets after "keep other" mean neither person has any non-department groups. Both rows show the status DryRun / Not confirmed.

Step 2: Capture the before state

```
Get-ADPrincipalGroupMembership amehta | Select-Object Name
```

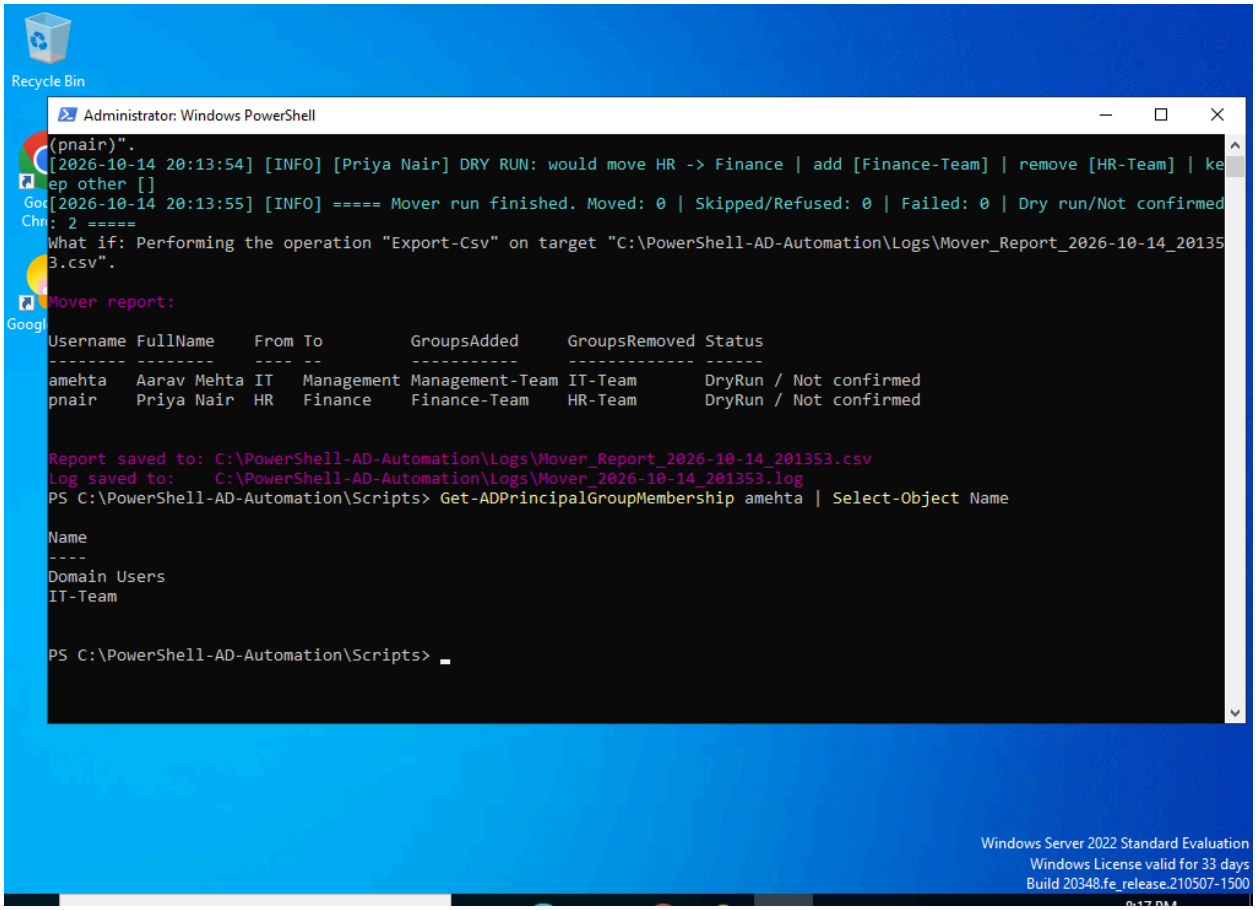


Figure 13: Aarav Mehta's groups before the move.

Aarav is in two groups: Domain Users, which every account has, and IT-Team.

Step 3: Execute

```
.\Invoke-Mover.ps1
```

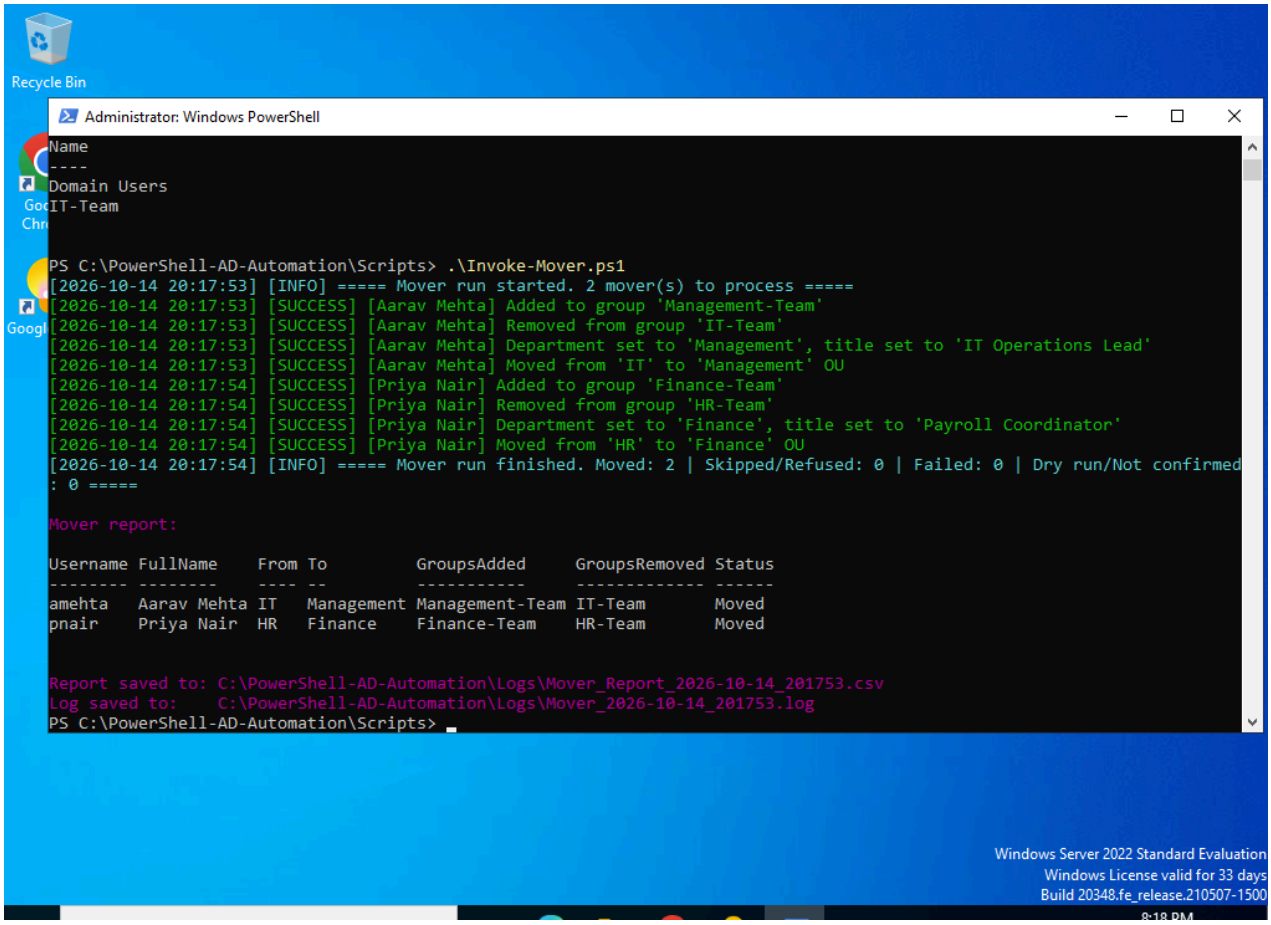


Figure 14: Real run. New groups added, old groups removed, titles and OUs updated.

For Aarav the log shows him added to Management-Team, removed from IT-Team, his department set to Management with the title IT Operations Lead, and the account moved from the IT OU to the Management OU. Priya gets the same sequence: added to Finance-Team, removed from HR-Team, department set to Finance with the title Payroll Coordinator, and moved from HR to Finance. The summary reads Moved: 2 | Skipped/Refused: 0 | Failed: 0.

Step 4: Verify the after state

```
Get-ADUser amehta -Properties Department, Title | Select-Object Name, Department, Title, DistinguishedName
Get-ADPrincipalGroupMembership amehta | Select-Object Name
```

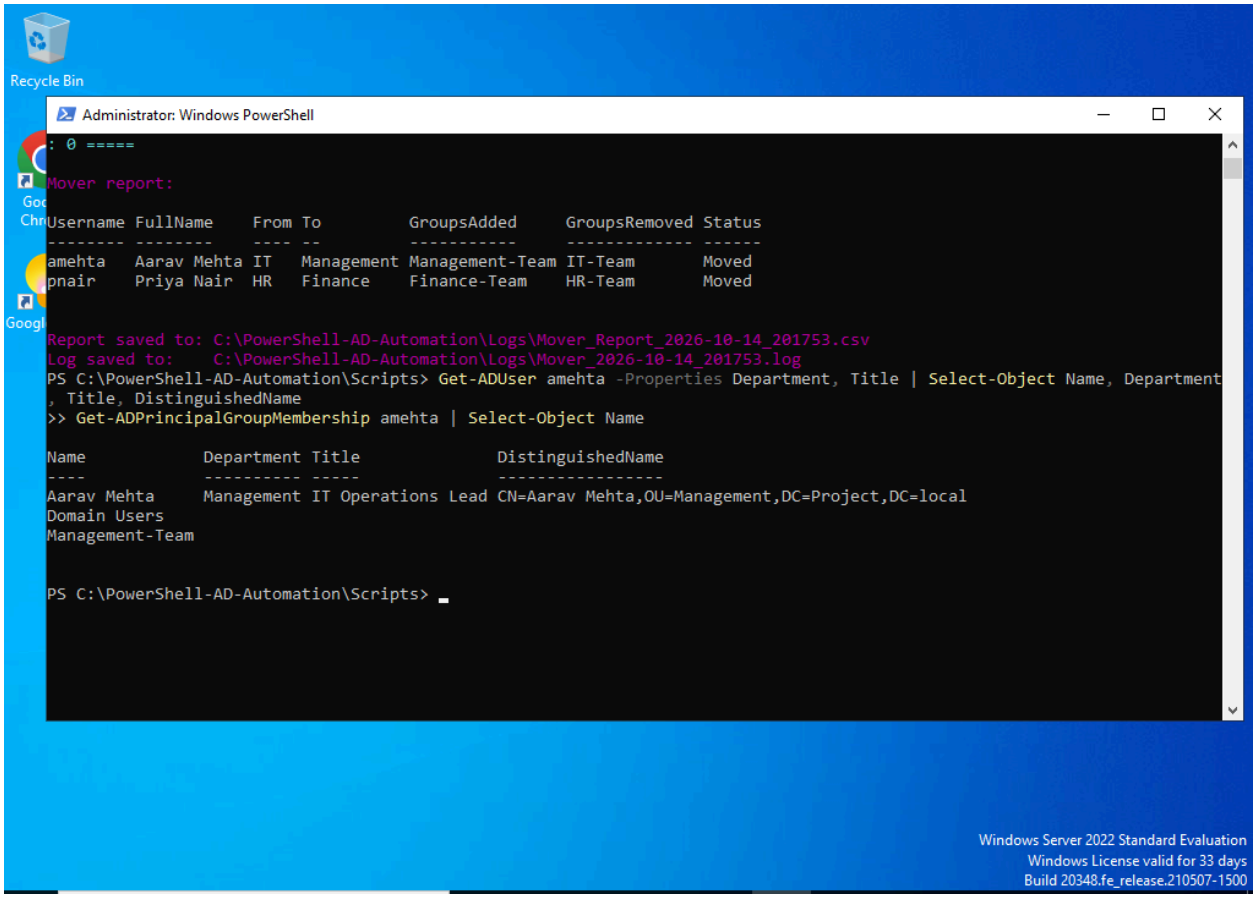


Figure 15: After the move. Aarav Mehta has a new department, title, OU and group.

Aarav's department is now Management, his title is IT Operations Lead, his account sits in the Management OU, and his groups are Domain Users and Management-Team. IT-Team is gone, which is exactly what the Mover script is supposed to guarantee.

Step 5: Re-run

```
.\Invoke-Mover.ps1
```

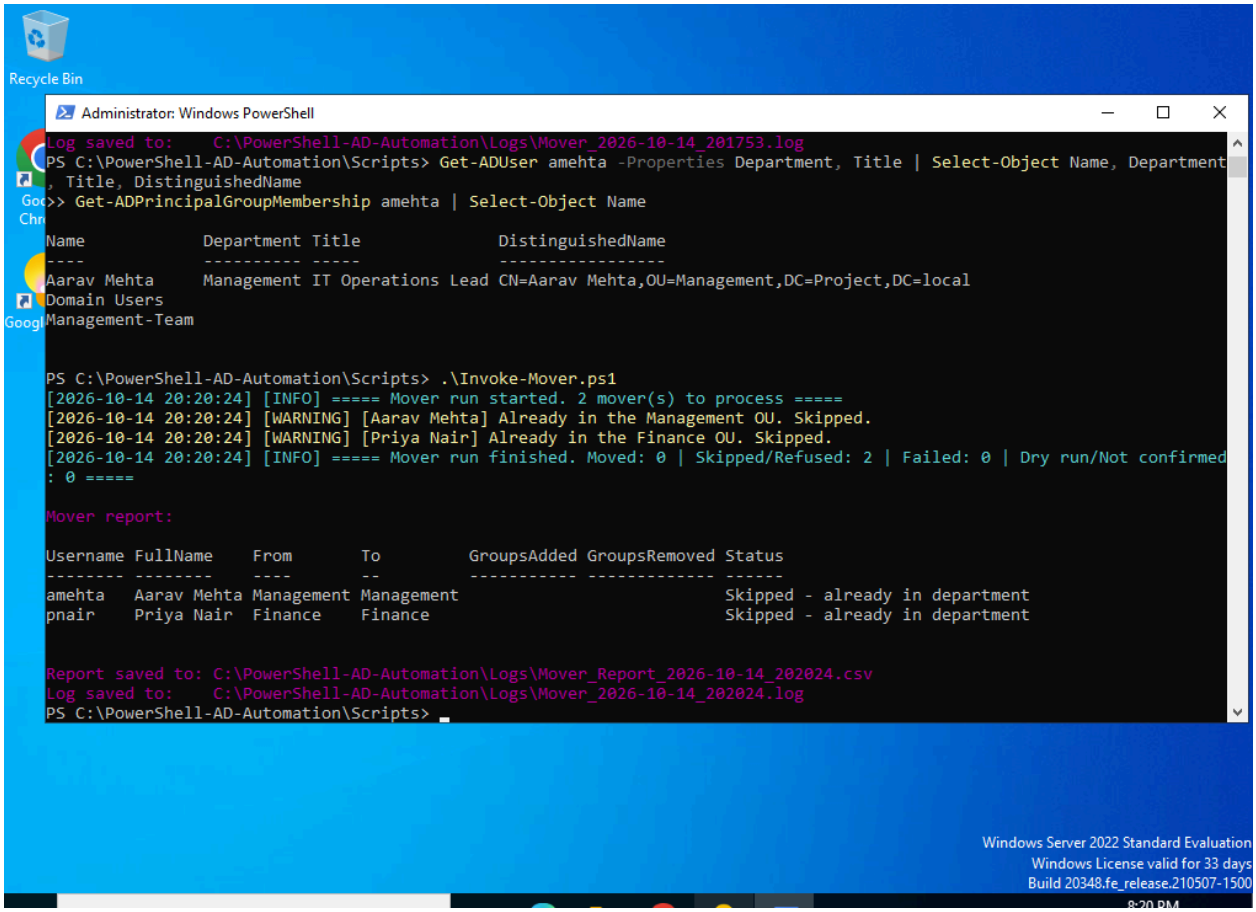


Figure 16: Re-running the Mover script. Both users are skipped because they are already in their new departments.

The log warns that Aarav is already in the Management OU and Priya is already in the Finance OU, and both rows show the status Skipped - already in department. Nothing was changed a second time.

Result and lessons

- Both users now have the access that matches their new department and nothing from the old one.
- Adding the new group before removing the old one means there is no moment when the person has no access.
- Non-department groups are never removed automatically. They are flagged for a manager to review, which is safer than guessing.
- The script is safe to run twice, which matters when a CSV gets re-sent.

What this log shows overall

- **Dry run first, every time.** Seeing the plan first means a mistake would show up before it could do any damage.
- **Verify twice.** I compared each script's report against Active Directory itself, in PowerShell and in Active Directory Users and Computers.
- **Capture the before state.** A before screenshot, or a report that records the previous OU and groups, is what turns a change into evidence.
- **Tickets tie each action to a request.** The ticket numbers sit in the CSV files and, for leavers, on the disabled accounts themselves.
- **I made typing mistakes along the way** (forgetting .\ in front of a script name, a wrong file extension). PowerShell's error messages were clear, and pressing Tab to complete script names avoids both.

The next document covers the audit reports and how these scripts behave when the input data is broken: Document 3, Audit and Validation Report.